



COMSATS UNIVERSITY ISLAMABAD

Department of Computer Engineering

Project Report: FPGA Based Image Processing

Course Name: Digital System Design

Student Name	Registration No.
1. M. Moiz Malik	FA23-BCE-073
2. Rohan Kiani	FA23-BCE-095

Submitted to: Mr. M. Bilal Qasim

Submission Date: 20-5-2026

Contents

1. Introduction	3
2. Literature Review.....	3
3. Design Methodology	4
3.1 Overview of System Architecture	4
3.2 Image Acquisition and Storage	4
3.3 Sliding Window Generation (Line Buffering)	4
3.4 Convolution Processing Engine	5
3.5 Output Frame Buffer (BRAM)	5
3.6 VGA Display Subsystem.....	5
3.7 Control Flow and Operation Sequence	5
Phase 1: Initialization	5
Phase 2: Buffering	6
Phase 3: Processing	6
Phase 4: Display	6
3.8 Design Considerations	6
3.9 Finite State Machine (FSM).....	7
3.10 Algorithmic State Machine (ASM) Chart.....	8
3.11 Architectural Block Diagram.....	9
4. Verilog Implementation.....	10
4.1 Top Module	10
Code:	10
.....	11
4.2 VGA Synchronization Module	12
Code:	12
4.3 Line Buffer Module	13
Code:	13
.....	14
4.4 Convolution Engine Module	14

Code:	14
.....	15
4.5 BRAM Memory Module	15
4.6 Testbench Module	16
5. Results and Discussion.....	16
5.1 Simulation Results	16
5.2 VGA Output Results.....	17
5.3 RTL Design:.....	17
5.4 Resource and Design Observations	18
6. Conclusion	18
7. References	19

1. Introduction

This project presents an FPGA-based image processing system that performs convolution filtering on a grayscale image and displays the processed output on a VGA monitor. The system is implemented using Verilog HDL and targets a Nexys2 FPGA development board.

The objective of the project is to demonstrate real-time image processing using hardware-based parallel computation. The system converts a stored grayscale image into a filtered output using a 3×3 convolution kernel and visualizes the result through VGA display signals.

The design highlights the use of FPGA resources such as Block RAM (BRAM), line buffers, and synchronous control logic to achieve efficient pixel-level processing.

2. Literature Review

Image processing on Field Programmable Gate Arrays (FPGAs) has gained significant attention due to the increasing demand for real-time, low-latency visual computation systems. Unlike software-based implementations on CPUs or GPUs, FPGA-based architectures enable deeply parallel execution at the pixel level, making them suitable for high-throughput image processing applications.

A widely adopted approach in FPGA image processing systems is the use of convolution-based filtering techniques. Convolution operations form the basis of many spatial domain filters such as smoothing, blurring, sharpening, and edge detection. Hardware implementations typically employ fixed-size kernels (commonly 3×3) due to their balance between computational complexity and hardware resource utilization.

To efficiently support streaming image data, most FPGA designs use line buffer architectures. Line buffers store a limited number of previous image rows and enable the construction of a sliding window required for convolution operations. This approach eliminates the need for full-frame buffering, which is often constrained by limited on-chip memory resources.

In addition to computation, real-time visualization of processed images is commonly achieved using VGA display controllers. VGA systems operate based on strict timing requirements, typically using a 25 MHz pixel clock for 640×480 resolution. The synchronization of pixel data with horizontal and vertical sync signals is a critical aspect of such systems.

Several prior FPGA-based implementations also explore external or serial image streaming techniques, where image data is transferred into FPGA memory through communication interfaces such as UART or SPI. While flexible, these methods introduce latency and synchronization challenges, particularly when interfacing with real-time display systems.

Another commonly reported limitation in existing FPGA image processing architectures is memory bandwidth and Block RAM (BRAM) capacity. Many designs require trade-offs between

image resolution, buffering strategy, and processing pipeline depth due to limited on-chip storage resources.

Motivated by these constraints, modern FPGA designs increasingly adopt fully on-chip pipelined architectures where the input image is preloaded into BRAM, processed through streaming convolution pipelines, and directly displayed via VGA controllers. This approach provides a balance between hardware efficiency, timing determinism, and design simplicity.

3. Design Methodology

3.1 Overview of System Architecture

The proposed system follows a streaming-based FPGA image processing architecture designed to perform real-time convolution on a grayscale image and display the processed output using a VGA interface. The design is organized as a pipelined dataflow system, where each module performs a dedicated function in the image processing chain.

The overall system is structured into five main functional blocks:

- Image memory (BROM/BRAM)
- Line buffer for window generation
- Convolution processing engine
- Output frame buffer (BRAM)
- VGA display controller

This modular architecture enables continuous pixel processing with minimal control overhead.

3.2 Image Acquisition and Storage

The input image is preloaded into Block RAM (BRAM) using an initialization file. The image is stored in a linear address space and accessed sequentially using an address counter.

This approach eliminates the need for external communication interfaces and ensures deterministic access latency, which is critical for synchronous FPGA design.

3.3 Sliding Window Generation (Line Buffering)

To enable convolution, the system constructs a 3×3 pixel window using a line buffer architecture. Three memory rows are maintained to store consecutive pixel lines.

At each clock cycle, incoming pixel data is shifted into the buffer structure, allowing continuous generation of overlapping windows without requiring full-frame storage.

This technique significantly reduces memory usage and is widely adopted in hardware image processing systems.

3.4 Convolution Processing Engine

The convolution engine performs spatial filtering using a fixed 3×3 kernel. Each pixel in the generated window is multiplied with the corresponding kernel coefficient, and the results are accumulated to produce a single output pixel value.

In this design, a uniform averaging kernel is used, effectively implementing a smoothing (blur) filter. The output is normalized and clipped to ensure 8-bit intensity range compliance.

This stage operates in a fully pipelined manner, enabling one output per valid window cycle.

3.5 Output Frame Buffer (BRAM)

The processed pixel stream is stored in a second BRAM acting as a frame buffer. A separate address generation mechanism assigns memory locations for each output pixel.

The memory operates in dual-mode:

- Write mode during convolution processing
- Read mode during VGA display

A control signal (done) is used to switch between processing and display phases.

3.6 VGA Display Subsystem

The VGA controller generates horizontal and vertical synchronization signals based on standard 640×480 timing at 60 Hz refresh rate. A 25 MHz pixel clock is used to maintain proper timing alignment.

During active video periods, pixel coordinates are mapped to BRAM addresses, allowing the processed image to be read and displayed in real time.

The image is rendered in RGB332 format, derived from the 8-bit grayscale output.

3.7 Control Flow and Operation Sequence

The system operates in three major phases:

Phase 1: Initialization

- Image is loaded into BROM/BRAM
- Address counters are reset

Phase 2: Buffering

- Line Buffers get filled enough
- Window valid is generated to start next phase

Phase 3: Processing

- Sliding window is generated continuously
- Convolution engine computes filtered pixels
- Output is stored into BRAM
- Address counter increments sequentially

Phase 4: Display

- Processing is halted using done signal
- VGA controller reads processed image from BRAM
- Image is displayed on monitor

3.8 Design Considerations

Several key constraints influenced the design methodology:

- Limited BRAM capacity required compact image resolution
- Streaming architecture was preferred over frame buffering
- Fully synchronous design was used to ensure timing reliability
- Pipeline-based computation was chosen to maximize throughput

3.9 Finite State Machine (FSM)

The proposed system is controlled using a finite state machine consisting of four primary states: Initialization, Buffering, Processing, and Display. During the initialization state, memory addressing and control signals are configured. The buffering state fills up the line buffers and signals if a window is valid to start next phase. The processing state performs image streaming, sliding-window generation, and convolution computation while storing filtered pixels into BRAM. Upon completion of processing, indicated by the done signal, the FSM transitions into the display state where VGA-based image visualization is continuously performed.

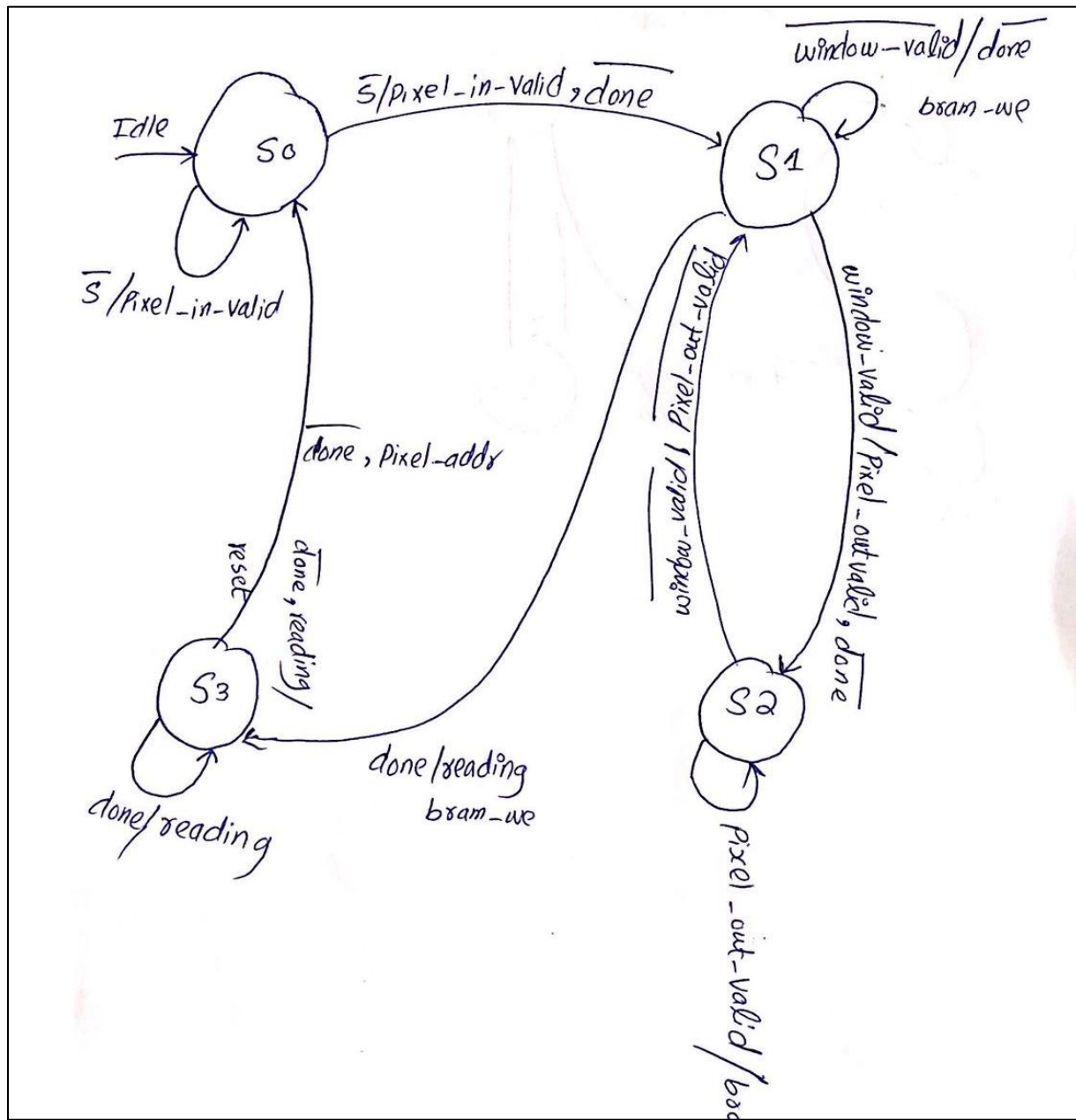


Figure 3.9: Finite State Machine

3.10 Algorithmic State Machine (ASM) Chart

The proposed design operates using an Algorithmic State Machine (ASM) structure consisting of initialization, buffering, processing, and display phases. The ASM controls image streaming from memory, sliding-window generation through line buffers, convolution computation, BRAM write operations, and VGA-based output display. Decision conditions such as `window_valid` and `done` determine state transitions and synchronization between processing and display stages.

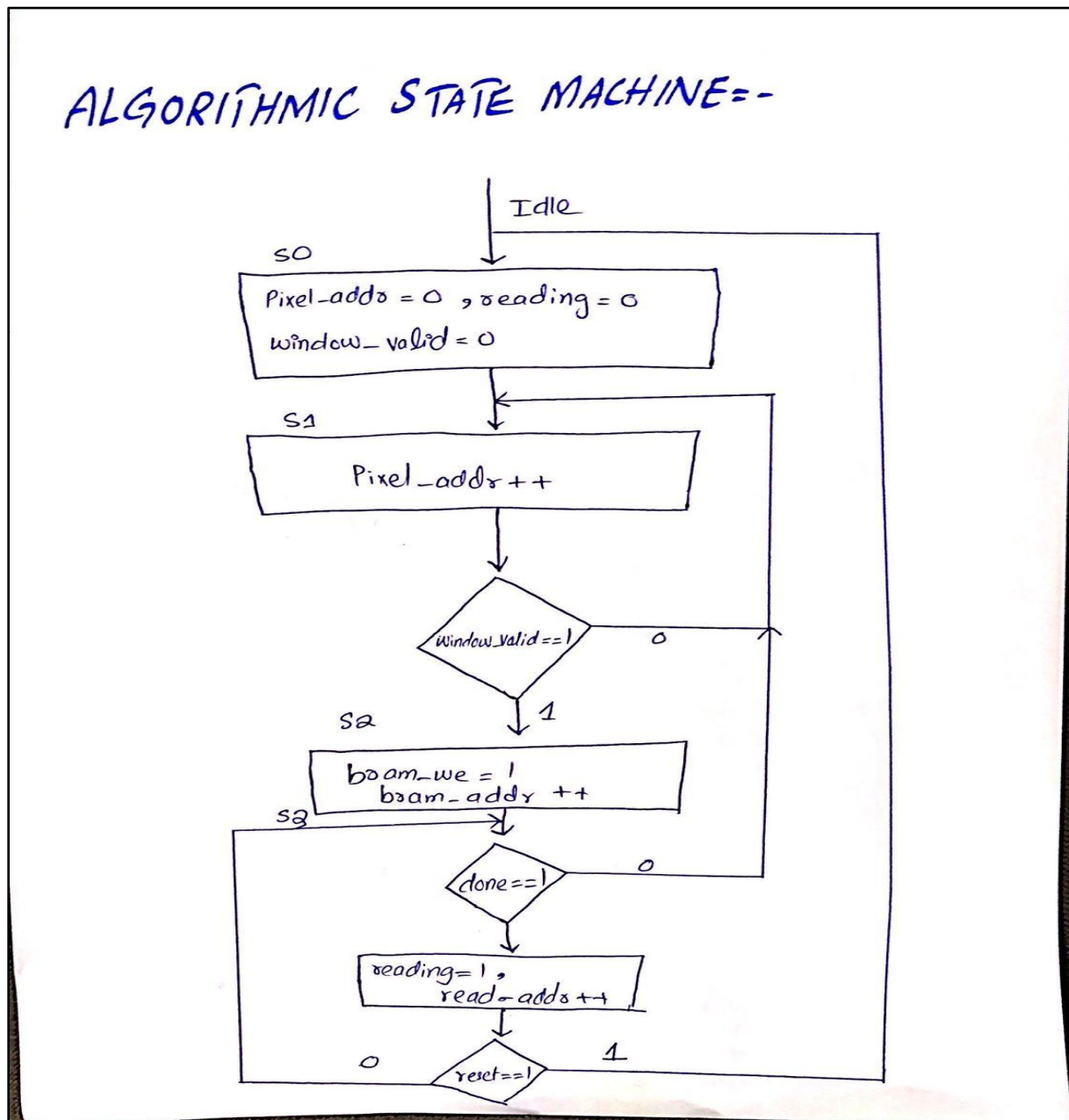


Figure 3.10: Algorithmic State Machine

3.11 Architectural Block Diagram

The proposed architecture consists of BROM, line buffer, convolution engine, BRAM, and VGA controller modules connected in a streaming pipeline. The input image is read from BROM and passed through the line buffer to generate a 3×3 sliding window. The convolution engine processes each window and stores the filtered output into BRAM. After processing completion, the VGA controller reads the processed image from BRAM and displays it on the monitor using VGA synchronization signals.

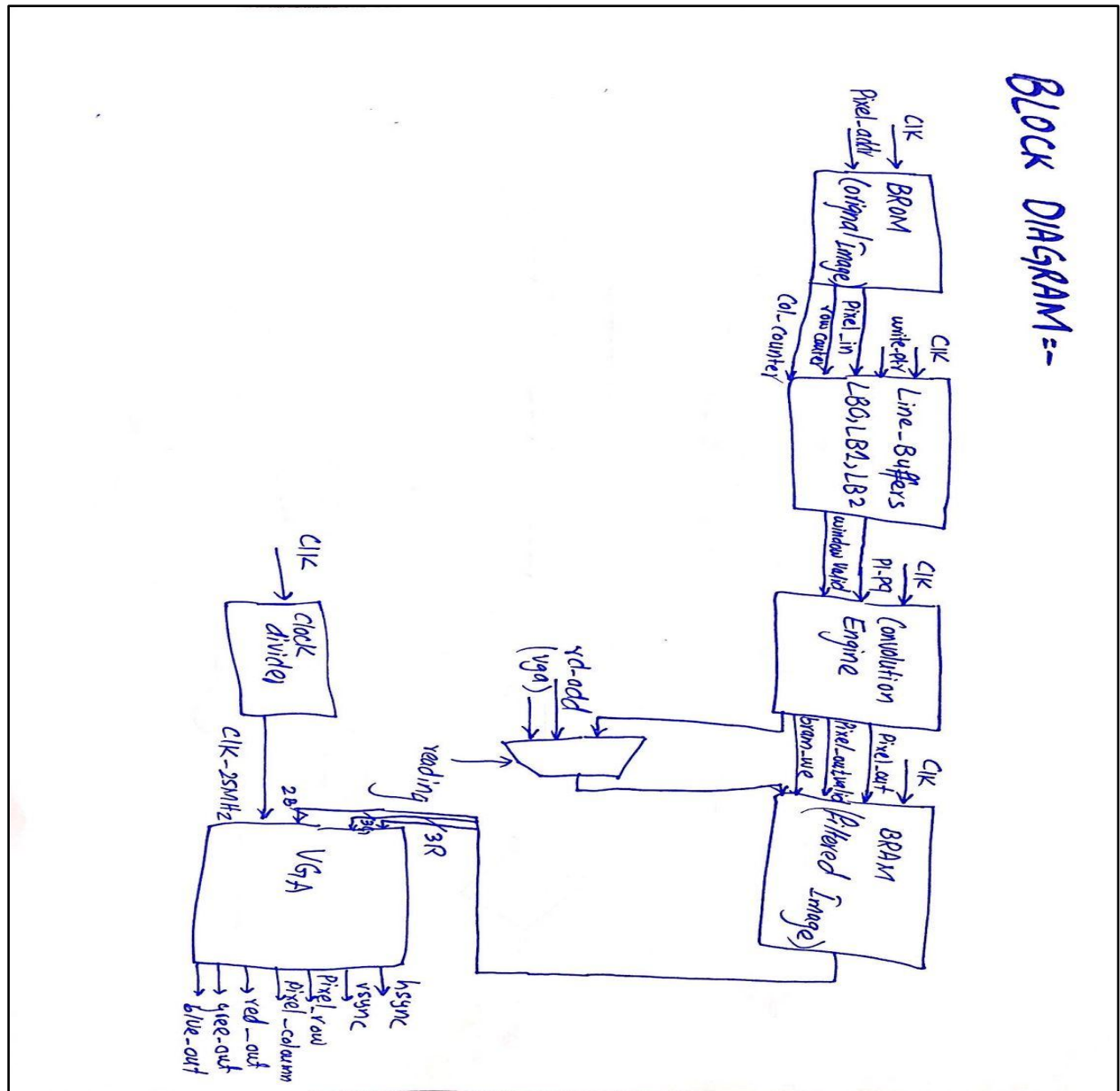


Figure 3.11: Block Diagram

4. Verilog Implementation

4.1 Top Module

The top_module acts as the central integration unit of the complete system. It connects all functional blocks including the BROM, line buffer, convolution engine, BRAM, and VGA controller. The module also generates the 25 MHz VGA clock using clock division and controls the switching between processing mode and display mode through the done signal. Address generation and BRAM control logic are also implemented in this module.

Code:

```
`timescale 1ns/1ps
module top_module( input wire clk,
output wire [2:0] red_out,
output wire [2:0] green_out,
output wire [1:0] blue_out, output wire hsync,
output wire vsync);
// CLOCK DIVIDE
reg clk_25mhz = 0;
always @(posedge clk)
    clk_25mhz <= ~clk_25mhz;
// INPUT ADDRESS COUNTER
reg [13:0] pixel_addr;
reg pixel_in_valid;
initial begin
    pixel_addr = 0;
    pixel_in_valid = 0;
end
always @(posedge clk) begin
    if (pixel_addr < 16384) begin
        pixel_addr <= pixel_addr + 1;
        pixel_in_valid <= 1;
    end
    else
        pixel_in_valid <= 0;
    end
end
// BROM
wire [7:0] pixel_data;
BROM brom0( .clka(clk), .addra(pixel_addr),
.douta(pixel_data));
reg pixel_in_valid_d;
always @(posedge clk)
    pixel_in_valid_d <= pixel_in_valid;
// LINE BUFFER
wire [7:0] P1.P2.P3.P4.P5.P6.P7.P8.P9:
```

```
wire window_valid;
// CONV ENGINE
wire [7:0] pixel_out;
wire pixel_out_valid;
wire done;
wire [13:0] ce_bram_addr;
conv_engine CE
(.clk(clk),dow_valid(window_valid),
.P1(P1), .P2(P2), .P3(P3),.P4(P4), .P5(P5),
.P6(P6),.P7(P7), .P8(P8), .P9(P9),
.pixel_out(pixel_out),.pixel_out_valid(pixel_o
ut_valid),.done(done),.bram_addr_out(ce_br
am_addr));
// VGA ADDRESS GENERATION
wire [9:0] pixel_row;
wire [9:0] pixel_col;
reg [13:0] vga_addr;
always @(*) begin
    if (pixel_row < 128 && pixel_col < 128)
        vga_addr = pixel_col * 128 + pixel_row;
    else
        vga_addr = 0;
end
// BRAM ADDRESS PIPELINE
reg [13:0] vga_addr_d;
always @(posedge clk_25mhz)
    vga_addr_d <= vga_addr;
// BRAM CONTROL
wire [13:0] bram_addr;
assign bram_addr = (!done) ? ce_bram_addr
: vga_addr_d;
wire bram_we = pixel_out_valid && !done;
```

```

/ BRAM
wire [7:0] bram_dout;
BRAM bram_inst(.clka(clk),.wea(bram_we),
.addra(bram_addr),.dina(pixel_out),
.douta(bram_dout));

// RGB332 OUTPUT
wire [2:0] red;
wire [2:0] green;
wire [1:0] blue;

assign red  = done ? bram_dout[7:5] : 0;
assign green = done ? bram_dout[7:5] : 0;
assign blue  = done ? bram_dout[7:6] : 0;

// VGA MODULE
vga_sync
VGA(.clock_25MHz(clk_25mhz),.red(red),.green(green),.blue(blue), .red_out(red_out),
.green_out(green_out),.blue_out(blue_out),
.hsycn(hsync),.vsync(vsync),.pixel_row(pixel_row),.pixel_col(pixel_col));

endmodule

```

4.2 VGA Synchronization Module

The vga_sync module generates the VGA timing signals required for image display. Horizontal and vertical counters are used to produce HSYNC and VSYNC signals according to standard 640×480 VGA timing. The module also generates pixel row and column coordinates for memory addressing. RGB outputs are enabled only during the active video region.

Code:

```
`timescale 1ns/1ps
module vga_sync( input wire clock_25MHz,
  input wire [2:0] red,input wire [2:0] green,
  input wire [1:0] blue,output reg [2:0] red_out,
  output reg [2:0] green_out,output reg [1:0] blue_out,
  output reg hsync,output reg vsync,
  output reg [9:0] pixel_row,output reg [9:0] pixel_col );
reg [9:0] h_cnt = 0;
reg [9:0] v_cnt = 0;
reg video_on;
always @(posedge clock_25MHz) begin
  // HORIZONTAL COUNTER
  if (h_cnt == 799)
    h_cnt <= 0;
  else
    h_cnt <= h_cnt + 1;
  // VERTICAL COUNTER
  if (h_cnt == 799) begin
    if (v_cnt == 524)
      v_cnt <= 0;
    else
      v_cnt <= v_cnt + 1;
  end
  // HSYNC
  hsync <= ~(h_cnt >= 656) && (h_cnt < 752));
  // VSYNC
  vsync <= ~(v_cnt >= 490) && (v_cnt < 492));
  // VIDEO ENABLE
  video_on <= (h_cnt < 640) && (v_cnt < 480);
  // CURRENT PIXEL LOCATION
  pixel_col <= h_cnt;
  pixel_row <= v_cnt;
```

```
// RGB OUTPUTS
if(video_on) begin
  red_out <= red;
  green_out <= green;
  blue_out <= blue;
end
else begin
  red_out <= 0;
  green_out <= 0;
  blue_out <= 0;
end
end
endmodule
```

4.3 Line Buffer Module

The line_buffer module generates the 3×3 sliding window required for convolution processing. Three line buffers are used to temporarily store consecutive rows of image pixels. As new pixels arrive sequentially, neighboring pixel groups are continuously formed and provided to the convolution engine. The module asserts the window_valid signal once a complete processing window becomes available.

Code:

```
`timescale 1ns/1ps
module line_buffer( input wire clk, input wire rst,
    input wire [7:0] pixel_in, input wire pixel_in_valid,
    output reg [7:0] P1,P2,P3,
    output reg [7:0] P4,P5,P6,
    output reg [7:0] P7,P8,P9,
    output reg window_valid);

reg [7:0] LB0 [0:127];
reg [7:0] LB1 [0:127];
reg [7:0] LB2 [0:127];
reg [6:0] col_counter;
reg [6:0] row_counter;
reg [1:0] write_ptr;
always @(posedge clk) begin
    if (rst) begin
        col_counter <= 0;
        row_counter <= 0;
        write_ptr <= 0;
        window_valid <= 0;
    end
    else if (pixel_in_valid) begin
        // WRITE PIXEL
        case(write_ptr)
            0: LB0[col_counter] <= pixel_in;
            1: LB1[col_counter] <= pixel_in;
            2: LB2[col_counter] <= pixel_in;
        endcase
        // GENERATE WINDOW
        if (row_counter >= 2 && col_counter >= 2)
            begin
```

```
0: begin
            P1 <= LB1[col_counter-2];
            P2 <= LB1[col_counter-1];
            P3 <= LB1[col_counter];
            P4 <= LB2[col_counter-2];
            P5 <= LB2[col_counter-1];
            P6 <= LB2[col_counter];
            P7 <= LB0[col_counter-2];
            P8 <= LB0[col_counter-1];
            P9 <= LB0[col_counter];
        end
        1: begin
            P1 <= LB2[col_counter-2];
            P2 <= LB2[col_counter-1];
            P3 <= LB2[col_counter];
            P4 <= LB0[col_counter-2];
            P5 <= LB0[col_counter-1];
            P6 <= LB0[col_counter];
            P7 <= LB1[col_counter-2];
            P8 <= LB1[col_counter-1];
            P9 <= LB1[col_counter];
        end
        2: begin
            P1 <= LB0[col_counter-2];
            P2 <= LB0[col_counter-1];
            P3 <= LB0[col_counter];
            P4 <= LB1[col_counter-2];
            P5 <= LB1[col_counter-1];
            P6 <= LB1[col_counter];
            P7 <= LB2[col_counter-2];
            P8 <= LB2[col_counter-1];
            P9 <= LB2[col_counter];
        end
    end
endcase
```

```

window_valid <= 1'b1;
end
else begin
    window_valid <= 1'b0;
end
// COUNTERS
if (col_counter == 127) begin
    col_counter <= 0;
    row_counter <= row_counter + 1;
    if (write_ptr == 2)
        write_ptr <= 0;
    else
        write_ptr <= write_ptr + 1;
end
else begin
    col_counter <= col_counter + 1;
end
end
end
endmodule

```

4.4 Convolution Engine Module

The conv_engine module performs the convolution operation using the generated 3×3 pixel window. Each pixel is multiplied with the corresponding kernel coefficient and accumulated to produce the filtered output value. The design uses a uniform averaging kernel to implement a blur filter. The module also generates BRAM write addresses and asserts the done signal after all pixels have been processed.

Code:

```

`timescale 1ns/1ps
module conv_engine( input wire    clk,
    input wire    window_valid,
    input wire [7:0] P1,P2,P3,
    input wire [7:0] P4,P5,P6,
    input wire [7:0] P7,P8,P9,
    output reg [7:0] pixel_out,
    output reg    pixel_out_valid,
    output reg    done,
    output reg [13:0] bram_addr_out);
    reg [7:0] kernel [0:8];
    initial begin
        kernel[0]=1; kernel[1]=1; kernel[2]=1;
        kernel[3]=1; kernel[4]=1; kernel[5]=1;
        kernel[6]=1; kernel[7]=1; kernel[8]=1;
    end
end

```

```

    reg [6:0] out_row;
    reg [6:0] out_col;
    reg [19:0] sum;

    initial begin
        out_row    = 7'd1;
        out_col    = 7'd1;

        done       = 1'b0;

        pixel_out   = 8'd0;
        pixel_out_valid = 1'b0;

        bram_addr_out = 14'd129;
    end
end

```

```

always @(posedge clk) begin
    pixel_out_valid <= 1'b0;
    if (window_valid && !done) begin

        // convolution sum
        sum <= (P1*kernel[0]) + (P2*kernel[1]) +
(P3*kernel[2]) +
        (P4*kernel[3]) + (P5*kernel[4]) +
(P6*kernel[5]) +
        (P7*kernel[6]) + (P8*kernel[7]) +
(P9*kernel[8]);
        // divide by 8 and clamp
        if ((sum >> 3) > 255)
            pixel_out <= 8'd255;
        else
            pixel_out <= sum[10:3];
        // store address
        bram_addr_out <= (out_row * 14'd128) +
out_col;
        pixel_out_valid <= 1'b1;
        // done condition
        if (out_row == 7'd126 && out_col ==
7'd126)
            done <= 1'b1;
        // counter update
        else if (out_col == 7'd126) begin
            out_col <= 7'd1;
            out_row <= out_row + 7'd1;
        end
        else begin
            out_col <= out_col + 7'd1;
        end
    end
end
endmodule

```

4.5 BRAM Memory Module

The BRAM module stores the filtered output image generated by the convolution engine. During processing, BRAM operates in write mode and stores processed pixels sequentially. After completion, the VGA controller reads pixel values from BRAM for display purposes. This dual-purpose memory structure reduces resource utilization and simplifies data management.

4.6 Testbench Module

The tb_top module is used to verify the functionality of the complete design through simulation. It generates the system clock, waits for the convolution process to complete, and reads back processed pixel data from BRAM. The filtered output values are written into a text file for analysis and verification. Monitoring statements are also included to observe important internal signals during simulation.

5. Results and Discussion

5.1 Simulation Results

The simulation results verified the correct operation of the proposed image processing pipeline. The line buffer successfully generated valid 3×3 sliding windows, indicated through the window_valid signal. The convolution engine produced filtered output pixels sequentially and stored them into BRAM using generated write addresses. The pixel_out_valid signal confirmed valid output generation, while the done signal correctly indicated the completion of the convolution process.

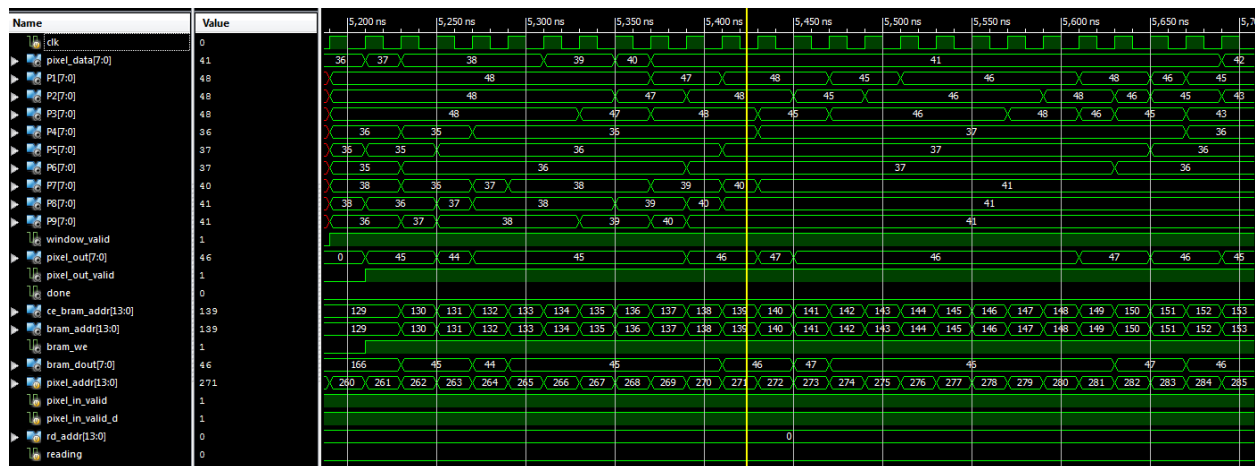


Figure 5.1: Simulation waveform showing convolution processing and BRAM write operation.

5.2 VGA Output Results

The VGA controller successfully generated horizontal and vertical synchronization signals according to standard 640×480 VGA timing requirements. After completion of image processing, the VGA subsystem continuously read processed pixel data from BRAM and displayed the filtered image on the monitor. The displayed output confirmed the successful implementation of the grayscale blur filtering operation.



Figure 5.2: VGA display output showing filtered grayscale image.



Figure 5.3: Original Image

5.3 RTL Design:

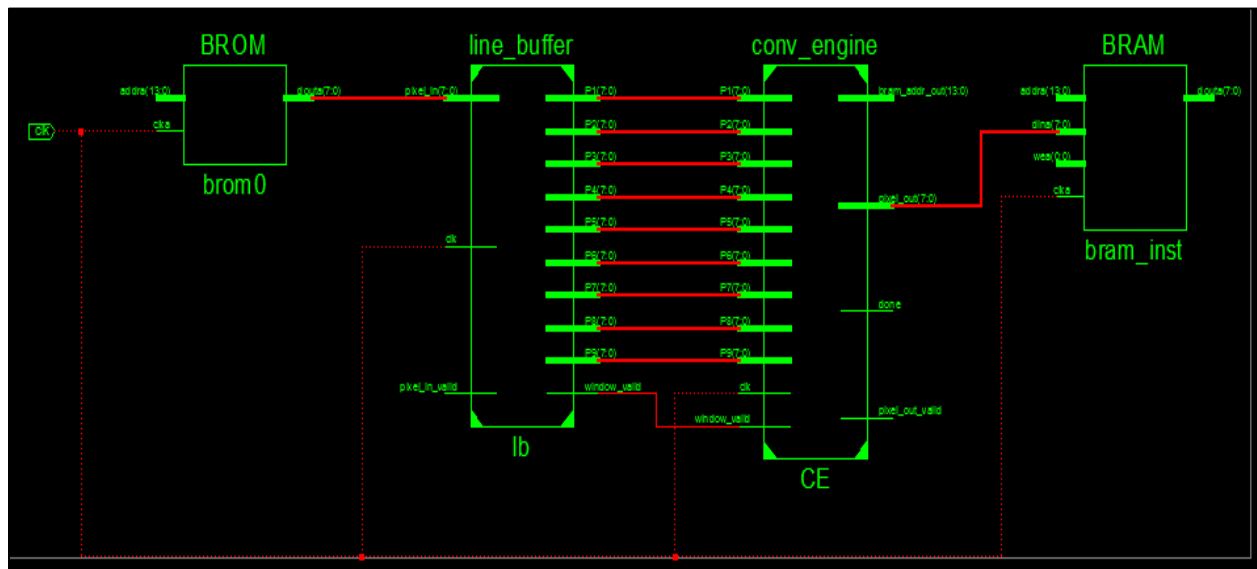


Figure 5.4: RTL schematic of the implemented FPGA image processing architecture.

5.4 Resource and Design Observations

The design was implemented considering the limited BRAM resources available on the FPGA board. To reduce memory utilization, a compact grayscale image resolution was selected and a streaming-based line buffer architecture was used instead of full-frame buffering. The modular pipeline structure simplified synchronization between image processing and VGA display operations while maintaining continuous pixel-level processing.

Design Overview - Summary - IOB Properties - Module Level Utilization - Timing Constraints - Pinout Report - Clock Report - Static Timing Errors and Warnings - Parser Messages - Synthesis Messages - Translation Messages - Map Messages - Place and Route Messages - Timing Messages - Bitgen Messages - All Implementation Messages Detailed Reports - Synthesis Report - Translation Report - Map Report - Place and Route Report - Post-PAR Static Timing Report - Power Report - Bitgen Report Secondary Reports - ISIM Simulator Log Design Properties - Enable Message Filtering Optional Design Summary Contents - Show Clock Report - Show Failing Constraints - Show Warnings - Show Errors	top_module Project Status (05/18/2026 - 11:32:12)			
	Project File:	cep.xise	Parser Errors:	No Errors
	Module Name:	top_module	Implementation State:	Synthesized
	Target Device:	xc3s500e-4fg320	Errors:	No Errors
	Product Version:	ISE 14.5	Warnings:	76 Warnings (5 new)
	Design Goal:	Balanced	Routing Results:	
	Design Strategy:	Xilinx Default (unlocked)	Timing Constraints:	
	Environment:	System Settings	Final Timing Score:	

Device Utilization Summary (estimated values)			
Logic Utilization	Used	Available	Utilization
Number of Slices	1055	4656	22%
Number of Slice Flip Flops	514	9312	5%
Number of 4 input LUTs	1660	9312	17%
Number of bonded IOBs	11	232	4%
Number of BRAMs	16	20	80%
Number of GCLKs	2	24	8%

Detailed Reports					
Report Name	Status	Generated	Errors	Warnings	Infos
Synthesis Report	Current	Mon May 18 11:32:10 2026	0	76 Warnings (5 new)	22 Infos (0 new)
Translation Report	Out of Date	Thu May 14 21:48:32 2026	0	0	0
Map Report	Out of Date	Thu May 14 21:48:40 2026		1 Warning (1 new)	3 Infos (3 new)
Place and Route Report					
Power Report					
Post-PAR Static Timing Report					
Bitgen Report					

Figure 5.5: Resources Used

6. Conclusion

This project successfully implemented an FPGA-based image processing system capable of performing real-time convolution filtering and VGA image display. The design utilized a line-buffer-based sliding window architecture, a 3×3 convolution kernel, BRAM memory structures, and a VGA synchronization controller to achieve efficient hardware-based image processing. The processed grayscale image was successfully displayed on a VGA monitor after convolution completion. The project also demonstrated efficient utilization of limited FPGA memory resources through a streaming-based processing architecture. Future improvements may include implementation of advanced filtering kernels, higher image resolutions, and external image acquisition interfaces.

7. References

1. S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3rd ed., McGraw-Hill, 2014.
2. [Digital Design of Signal Processing Systems: A Practical Approach - Shoab Ahmed Khan - Google Books](#)
3. R. Gonzalez and R. Woods, *Digital Image Processing*, 4th ed., Pearson Education, 2018.
4. Xilinx Inc., *Block Memory Generator v8.4 Product Guide*, Available: Xilinx Block Memory Generator Guide
5. TinyVGA, “VGA Signal Timing Information,” Available: TinyVGA VGA Timing Reference